

TP INTEGRADOR - ACTIVIDAD 4 - GRUPO 1



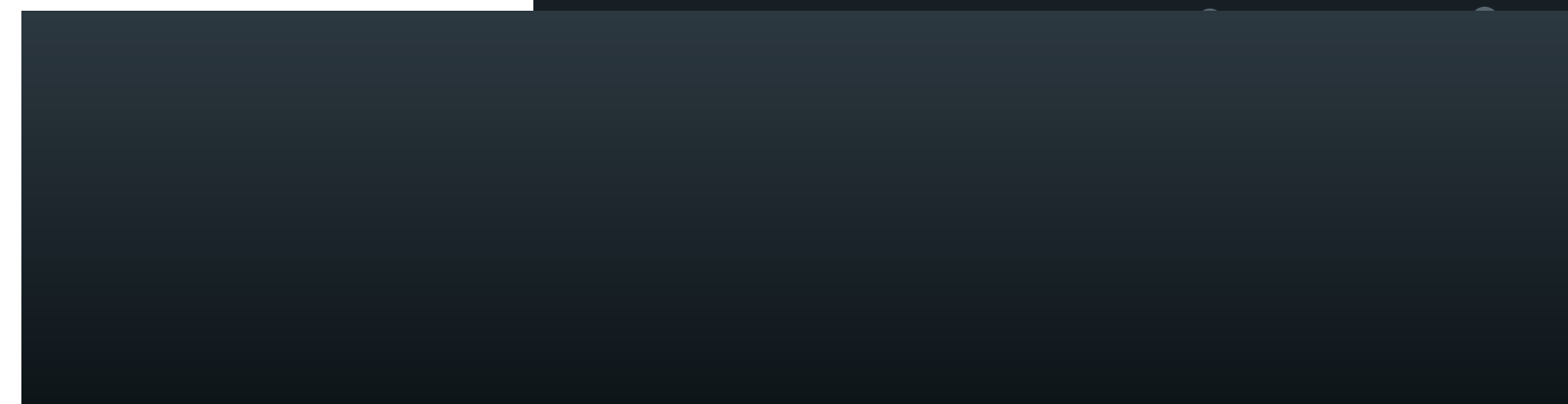
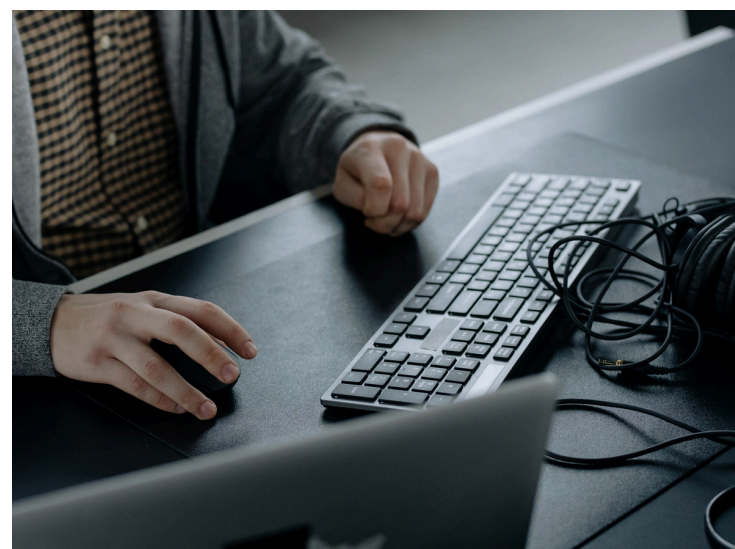
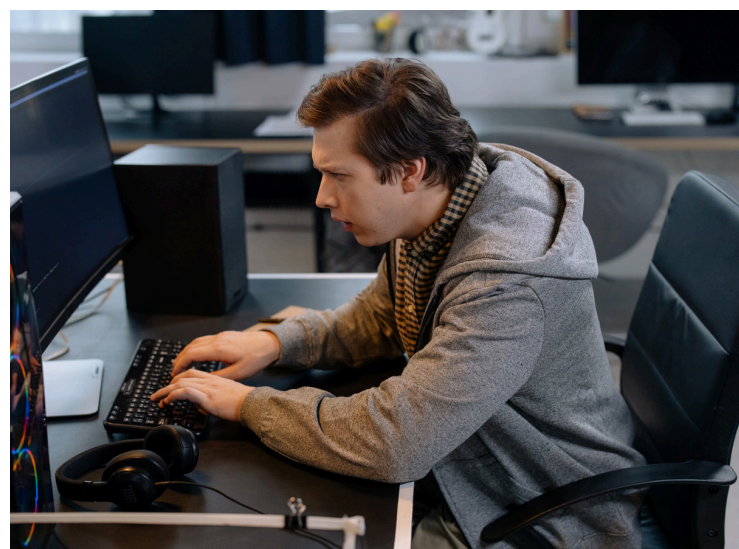
INGENIERÍA Y CALIDAD DE SOFTWARE

Integrantes: Schneeberger Angeles, Zacarias
Paula, Cemino Conrado, Castaño Rodrigo y
Cortés Matías.

Start Slide Now

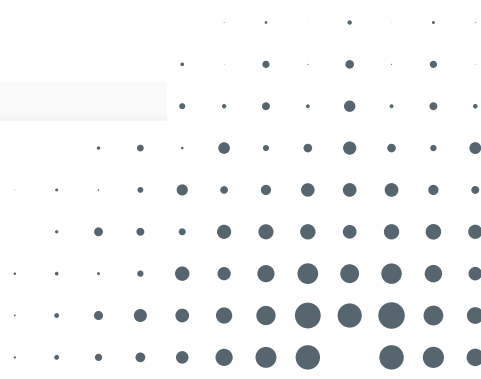


COMPARACIÓN DE RESULTADOS



VERIFICACIÓN TÉCNICA (ANTES Y DESPUÉS)

Métrica	Antes (desarrollo)	Después (producción)	Mejora
Tamaño imagen API	421 MB	165 MB	61%
Tamaño imagen Web	220MB	23.3MB	89%
Tiempo de startup API	0m19.198s	0m13.528s	29%
Memoria API (idle)	107.6 MB (sin límite)	72 MB / 512 MiB	33%
Memoria Web (idle)	376.7MiB / 6.649GiB	12.72MiB / 256MiB	96%
Endpoint API accesible	✓ 200 OK	✓ 200 OK	—
Frontend vía nginx	—	✓ Documento HTML*	—





```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>web</title>
    <script type="module" crossorigin src="/assets/index-BJA5EXvJ.js"></script>
  </head>
  <body>
    <div id="root"></div>
  </body>
</html>
```

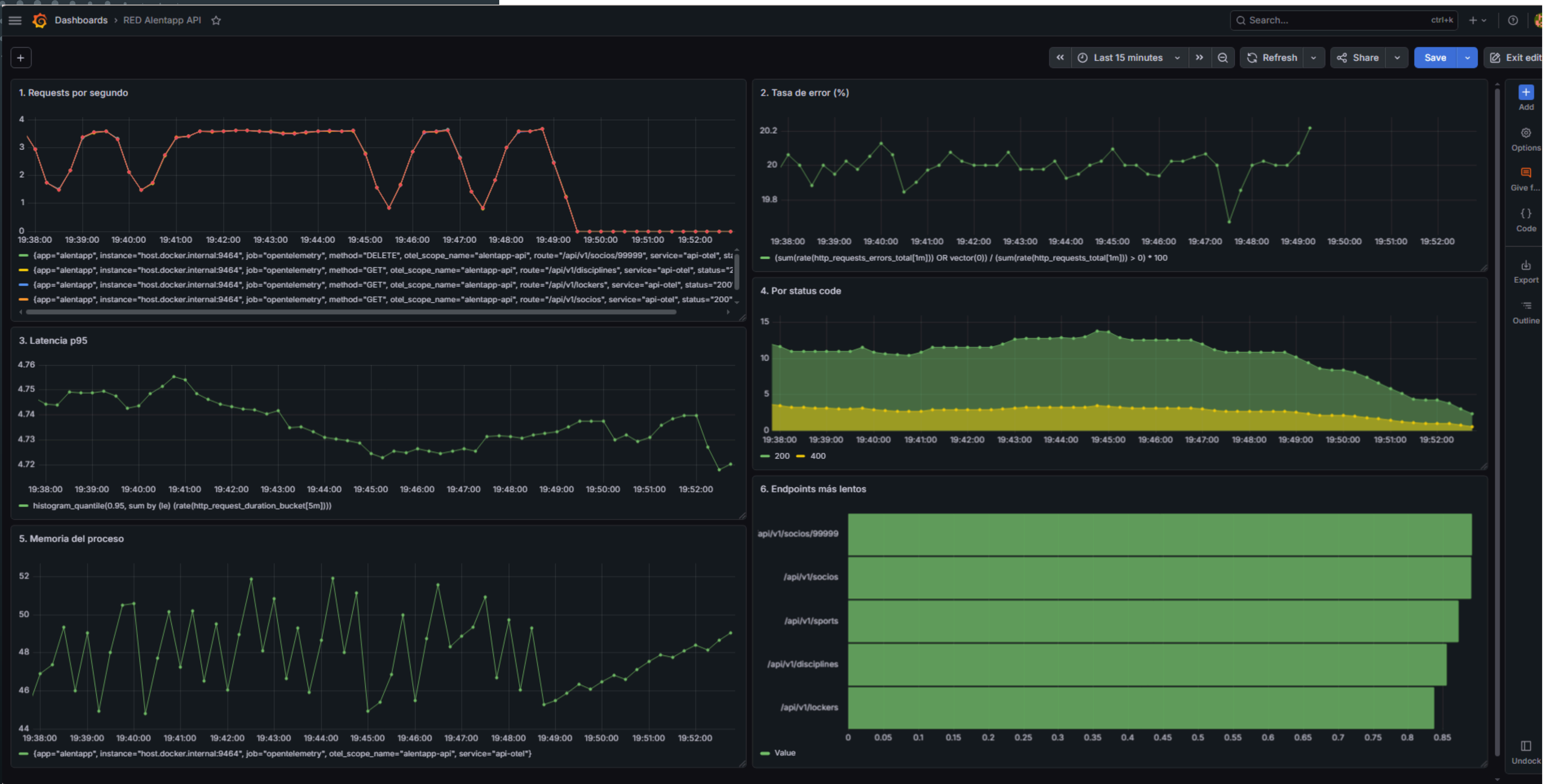
Verificación de seguridad

Medida de seguridad	Estado	Verificación
API corre con usuario no-root	✓ OK	<code>docker exec alentapp-api-prod whoami</code> → <code>node</code>
Web corre con usuario no-root	✓ OK	<code>docker exec alentapp-web-prod whoami</code> → <code>nginx</code>
npm ausente en imagen final de la API	✓ OK	<code>docker run --rm --entrypoint which alentapp-api:prod npm</code> → vacío
tsc ausente en imagen final de la API	✓ OK	<code>docker run --rm --entrypoint which alentapp-api:prod tsc</code> → vacío
Node presente en imagen de la API	✓ OK	<code>docker run --rm --entrypoint which alentapp-api:prod node</code> → <code>/usr/local/bin/node</code>
Filesystem read-only activo	✓ OK	<code>docker exec alentapp-api-prod touch /test</code> → falla
Filesystem read-only activo	✓ OK	<code>docker exec alentapp-web-prod touch /test</code> → falla
Variables sensibles vía .env	✓ OK	no hardcodeadas en Dockerfile ni docker-compose
Capabilities mínimas (cap_drop: ALL)	✓ OK	configurado en docker-compose.prod.yml
Healthchecks funcionando	✓ OK	<code>docker ps</code> → los 3 contenedores en estado <code>healthy</code>

Razones por las cuales se tomaron las medidas de seguridad:

- **Usuario no-root:** Si un atacante explota una vulnerabilidad en la app, no obtiene *permisos de administrador* dentro del contenedor.
- **Sin npm ni tsc:** Un atacante *no puede descargar ni ejecutar código malicioso* desde adentro del contenedor.
- **Filesystem read-only:** Si alguien entra al contenedor, que *no pueda escribir archivos* ni dejar malware.
- **Variables vía .env:** Las *credenciales no están en el código* fuente. Si el repositorio es público o se filtra, las contraseñas no quedan expuestas.
- **cap_drop: ALL:** Que el proceso *no tenga permisos especiales* del sistema operativo que no necesita.
- **Healthchecks:** Que Docker sepa cuándo un *servicio se rompió* y pueda reiniciarlo solo.

DASHBOARD DE OBSERVABILIDAD (METRICAS RED)



COMPORTAMIENTOS CON TRÁFICO

Requests por segundo

Incremento sostenido de peticiones hacia la API durante la ejecución de scripts.

Tasa de error (%)

El script de estrés enviaba 1 error por cada ciclo de 5 peticiones totales (4 éxitos y 1 fallo), generando una tasa de fallo que se estabilizó en el 20%.

Latencia p95

El 95% de los usuarios experimentaron tiempos de respuesta óptimos.

Por status code

Base dominante de peticiones exitosas (códigos 200/201/204) y superpuesta la franja correspondiente a los errores (códigos 400/404).

Memoria del proceso

El consumo de RAM no se disparó sin control ni colapsó el servidor, se mantuvo en niveles normales y equilibrados.

Script utilizado

```
for i in { 1 .. 300 }; do
  # Tráfico sano (Éxitos 200)
  curl -s http://localhost:3000/api/v1/socios > /dev/null
  curl -s http://localhost:3000/api/v1/sports > /dev/null
  curl -s http://localhost:3000/api/v1/lockers > /dev/null
  curl -s http://localhost:3000/api/v1/disciplines > /dev/null

  # Tráfico roto (DELETE a un ID que no existe -> Error 400 real)
  curl -X DELETE -s http://localhost:3000/api/v1/socios/99999 > /dev/null

  sleep 0.05
done
```

Endpoints más lentos

Genera un top 5 en formato de tabla de los múltiples endpoints que fueron sometidos a la prueba de carga de forma simultánea.

APRENDIZAJES



¿Qué fue lo más difícil?

- Aprender a separar las etapas de compilación y ejecución en Docker, y lograr pasar solo los archivos necesarios de una etapa a otra sin romper la aplicación.
 - Resolver las saturaciones de renderizado en los paneles para lograr una visual comprensible.
 - Manejo de valores nulos en PromQL para que los paneles no muestren "No data" cuando la API operaba de forma 100% sana.
-

¿Ventajas?

- Con Multistage se obtienen contenedores mas livianos, más velocidad y seguridad.
- Transición de un sistema a ciegas a uno con visibilidad total, permitiendo el análisis de rendimiento (latencia y errores) en tiempo real mediante el dashboard.
- Al ser una prueba de carga controlada se podía mantener un mayor supervisión y forzar fallos en los controladores más fácilmente.



THANK YOU

